Laborator 9 - Testarea si asigurarea calitatii unui proiect

Termeni

Bug - O problemă, eroare, defect, posibilitate nedocumentată, greșeală într-un sistem software care împiedică sistemul să funcționeze așa cum este de așteptat este un bug.

Bug report - Un raport care detaliază un bug într-un program, adică cum se manifestă, când, care parte a codului este responsabilă, eventual și o soluție posibilă.

Bug tracking - Bug tracking-ul este un mod organizat de a ține evidența bugurilor și a stărilor acestora (deschis, rezolvat, testat, închis). Acesta poate varia de la forme neorganizate (întâlniri în echipă) la forme organizate (liste de discuții, mail-uri, soluții specializate).

Sistem de bug tracking - O aplicație software care are ca rol ușurarea bug tracking-ului (este o metodă eficientă centralizată și ușor utilizabilă.) Există soluții gratuite, dar și soluții enterprise cu prețuri foarte ridicate. Un sistem de bug tracking este un subtip de sistem de issue tracking.

Sistem de issue tracking - O aplicație software care are ca rol centralizarea și adresarea tuturor cererilor legate de unul sau mai multe produse, de la probleme la schimbări de design și aplicări de patch-uri. Cele mai multe sunt în același timp și bug trackere. În general, sunt mai complexe decât bug trackerele.

Patch - O mică bucată de software creată spre a rezolva problemele și a îmbunătăți performanțele unui program. Aceasta include, dar nu este limitata la: repararea de bug-uri, creșterea vitezei, îmbunătățirea graficii. Unele patch-uri creează și probleme noi.

Patch management - Procesul de a stabili o strategie și a plănui ce patch-uri să fie aplicate, la ce sisteme și când este momentul potrivit pentru aplicarea lor.

Aplicații

GitHub

Așa cum a fost descris în cadrul laboratorului 4, GitHub [https://github.com/] permite și un management al bug-urilor.

Redmine

Redmine [http://www.redmine.org] este un tool complex, care oferă și suport pentru bug tracking. Prin secțiunea “New issue” se pot publica bug-uri, iar aceastea pot fi urmărite.

Bugzilla

Bugzilla [http://www.bugzilla.org/about/] este unul dintre cele mai populare sisteme de urmărire a bug-urilor având o multitudine de functionalitati [http://www.bugzilla.org/features].

Trac

Trac [http://trac.edgewall.org/] nu este atât de sofisticat ca Bugzilla în ce privește urmărirea bugurilor (adică bug tracking), dar oferă mult mai multe alte facilități precum un navigator de Subversion și un wiki, deci o platformă completă de colaborare între utilizatori și dezvoltatori. De asemenea Trac poate fi extins ușor prin intermediul modulelor [http://trac.edgewall.org/wiki/TracPlugins], unul dintre cele mai populare fiind cel pentru code review.

Debian bug tracking software

Fiecare bug are o adresă de mail distinctă. Comentariile și controlul bug-urilor se face cu emailuri tipizate.

Launchpad

Launchpad [https://launchpad.net] este o aplicație web și un site pentru publicarea proiectelor software (în special cele open source). Printre multe alte facilități interesante dispune și de bug-tracking [https://launchpad.net/+tour/bugs].

Launchpad poate centraliza rapoartele bugurile din mai multe surse independente și oferă posibilitatea dezvoltatorilor de a discuta despre acel bug într-un singur loc. Launchpad se integrază bine cu trackere externe: Bugzilla, Trac, Sourceforge, Roundup, Mantis, RT și Debian BTS. Launchpad încearcă să elimine patchurile scrise în comentarii prin oferirea posibilității de publicare a unei ramure de cod (branches). Bugurile pot fi urmărite prin email și atom feeds, fiecare bug având asociată o adresa de email (ex: 249177@bugs.launchpad.net).

Patch-uri

Un fișier patch e un fișier text care descrie diferențele dintre două versiuni ale unui fișier sau dintre două fișiere distincte. Există două formate standard pentru astfel de fișiere: tipul normal și context copiat. Mai jos este prezentat un format îmbunătățit, **Context unificat**, care este cel mai întâlnit și mai ușor de vizualizat format.

Structura unui fișier patch cu context unificat:

```
--- original_file_name comentarii (ștampilă de timp, versiunea fișierului în svn/git/etc., etc.)
+++ modified_file_name comentarii (idem)
@@ -start_line__original_file,nr_lines__original_file +start_line__modified_file,nr_lines__modified_file @@
context_line_before_1
context_line_before_2
   ...
context_line_before_n
-  original_line_1
-  original_line_2
   ...
-  original_line_m
+  modified_line_1
+  modified_line_2
   ...
+  modified_line_p
context_line_after_1
context_line_after_2
   ...
context_line_after_n
```

Exemplu de fișier patch cu context unificat:

```
--- v1.c       2008-11-18 02:34:38.000000000 +0200
+++ v2.c       2008-11-18 02:38:14.000000000 +0200
@@ -261,8 +261,18 @@
     if (build_key(argv[1]))
         return 2;

-    display("original  message: [%s]\n", msg, blocks);
-
+    //display("original  message: [%s]\n", msg, blocks);
+    if (blocks)
+    {
+        char c;
+        char * fmt = "original  message: [%s]\n";
+        int len = blocks;
+        c = msg[len-1];
+        if (c == 'X')
+            msg[len-1] = '\0';
+        printf(fmt, msg);
+        msg[len-1] = c;
+    }

     apply(msg, blocks, normalize);
     encrypt((unsigned short*) msg, blocks/2);
```

Creare patch

diff

'diff' e un program cu care se pot crea fișiere de tip patch având la dispoziție fișierul original și fișierul modificat. `diff` poate crea fișiere patch în oricare din cele trei formate standard:

- **normal**

```
diff origial_filename modified_filename
```

- **context unificat**

pentru a crea un fișier patch cu un număr implicit de linii de context `unificat`:

```
diff -u origial_filename modified_filename
```

pentru a crea un fișier patch cu x linii de context `unificat`:

```
diff -Ux origial_filename modified_filename
```

git format-patch

`git` poate genera fișiere de patch care descriu diferențele între două commit-uri. Implicit `git format-patch` crează un fișier diff în format unificat (cel mai răspândit format în lumea open-source) și îl scrie la stdout. Pentru a-l scrie într-un fișier pe disc, trebuie redirectată ieșirea către un fișier:

```
git format-patch [optional parameters] > file.patch
```

Utilizare patch

Un fișier `patch` nu este folosit doar pentru a vizualiza diferențele între două versiuni ale unor fișiere, ci și pentru a transmite un anumit set de modificări de la un utilizator la altul pentru a fi aplicate.

Utilitarul standard cu care se aplică patch-uri se numește `patch`.

Modul cel mai întâlnit de utilizare este:

```
patch -pNUM < filename.patch
```

`patch` își ia fișierul din stdin, de aceea trebuie redirectat fișierul de intrare.

În fișierul `filename.patch`, `diff` sau `git format-patch` va scrie și numele fișierelor pe care le-a comparat și din conținutul cărora a extras diferențele. E posibil ca `diff` sau `git format-patch` să fie rulate într-un alt director față de cel în care se va rula `patch`. Cu opțiunea `-pNUM` se specifică numărul de nivele din calea specificată în fișierul `filename.patch` care vor fi ignorate când se va încerca să se determine.

De asemenea, un patch poate fi aplicat cu comanda:

```
git am filename.patch
```

Test Cases

Pentru o testare de calitate, este important să existe o planificare riguroasă a testării încă din faza de proiectare sau development. Pe măsură ce se conturează definițiile modulelor, entităților de date, obiectelor, claselor, funcțiilor, etc. este recomandabil să se scrie și “scenarii” de testare ale acestora, fie *top-down*, fie *bottom-up*.

În industria software, “scenariile” de test se numesc **test cases**.

Un exemplu de test case:

trebuie verificată funcționarea unei pagini de login care conține un input de user name și unul de parolă

- click pe “Login” fără a completa user/pass → trebuie să rămân în pagină și să se afișeze un mesaj de eroare
- completat user corect, parola incorectă sau nulă, click pe “Login” → trebuie să rămân în pagină și să se afișeze un mesaj de eroare
- completat user corect, parola corectă, click pe “Login” → trebuie să fiu corect autentificat
- completat user corect, parola corectă, tastat “Enter” → trebuie să fiu corect autentificat

Există “testare pozitivă” și “testare negativă”, concretizată în **positive test cases** și **negative test cases**. Testarea pozitiva înseamnă *verificarea faptului că sistemul face ceea ce trebuie să facă*. Testarea negativă înseamnă verificarea faptului că sistemul *nu face ceea ce nu trebuie să facă*.

Pe cazul anterior:

- testare pozitivă: se verifică faptul că la user/pass corecte se face login iar la user/pass incorecte se afișează un mesaj de eroare
- testare negativă: se verifică faptul că la user/pass corecte NU se afișează un mesaj de eroare iar la user/pass incorecte NU se face login și NU se “sparge” sistemul

În principiu, cele două approachuri sunt echivalente, însă în practică testarea pozitivă se referă la funcționarea “normală” a sistemului, iar testarea negativă la “corner cases”. De exemplu, pentru testarea unui feature critic ca time to market dar non-critic ca și calitate (ex. twitter), se va prefera testarea pozitivă, care asigură că sistemul funcționează corect pentru cei mai mulți utilizatori. Pentru testarea unui feature critic ca și calitate (ex. online banking) se va insista pe teste negative, ex. se va încerca “spargerea” sistemului prin combinații incorecte.

Există, ca în orice alt domeniu, tool-uri open source pentru managementul test cases:

- Bugzilla Testopia: http://www.mozilla.org/projects/testopia/ [http://www.mozilla.org/projects/testopia/]
- Incremental Scenario Testing: https://sourceforge.net/projects/istt/ [https://sourceforge.net/projects/istt/]
- QA Traq: http://www.testmanagement.com/ [http://www.testmanagement.com/]

Black Box Testing

Se aplică în cazul în care pentru programul ce trebuie testat sursele nu sunt disponibile, ci doar interfața de acces (binarul, o interfața implementată de clasa testată, etc). Cum se testează: Se aplică un set de intrări, iar ieșirile sunt comparate cu un set de ieșiri corecte.

- Intrări - un set de intrări ce vor fi trimise programului pentru execuție. Intrările nu trebuie neapărat să fie niște fișiere ci pot fi niște apeluri de funcții sau un alt program ce simulează alte module ce interacționază cu programul testat.
- Ieșiri - pentru fiecare intrare se definește un set de ieșiri posibile cu care se testează ieșirea programului. În multe din cazuri ieșirea e deterministică și trebuie doar comparate ieșirea programului cu ieșirea așteptă, iar în alte cazuri este necesară scrierea unui program.

Exemplu Testarea unui program care la intrare primește un string pe care vrea să îl parseze în

- număr întreg
  - limitele inferioare și superioare
  - Unordered List Item câteva constante mici (0, -1, 1)
  - câteva constante mari
  - întregi negativi
  - caractere invalide (de exemplu, pentru baza 10 nu putem avea A)
  - numere care au 0 în față (dar excesiv de lungi gen: 00000000000000000000000000000666)
  - același lucru pentru diferite baze (dacă sunt suportate)
- float
  - ca mai sus, dar în principal trebuie ținut cont de valorile speciale definite în standardul IEEE 754 [http://ieeexplore.ieee.org/xpl/freeabs_all.jsp?arnumber=4610935]

        - -0.0 != 0.0
        - Nan, -Nan
        - inf, -inf
    - un șir de caractere
        - limbajul acceptat (LFA style)
    - un vector
        - se aplică cazurile de la numere întregi (pentru 1 element)
        - 0 elemente, 1 element, 2 elemente, 5 elemente, număr maxim de elemente
        - caractere de despărțire / elemente invalide

Boundary testing

Boundary testing sau boundary value analysis este o metodă de proiectare a suitelor de teste pentru cazurile în care se folosesc **valori la limită** acceptate de program. În general, accentuează testarea "corner case"-urilor. Unele teste care fac parte din suita boundary testing sunt "stress tests".

În general, boundary value analysis se realizează în doi pași: # identificarea claselor de echivalență # proiectarea suitelor de test

Primul pas înseamnă, de obicei, partiționarea valorilor posibile în clase valide și invalide. *Exemplu* Un program care primește valori pozitive până în 99, va avea trei clase:

- clasa validă (0 < n ⇐ 99)
- prima clasă invalidă (n ⇐ 0)
- a doua clasă invalidă (n > 99)

Al doilea pas înseamnă proiectarea unei suite de teste care vor selecta anumite valori care să verifice reacția programului la valori valide sau invalide. *Exemplu* Dacă un program primește valori în domeniul [-999,999], atunci o suită posibilă de test ar fi:

- testare valoare -999
- testare valoare 999
- testare valoare -998
- testare valoare 998

În general, testele trebuie să includă prima și ultima valoare posibilă. De asemenea, se recomandă testarea de condiții extreme de intrare sau ieșire (valori foarte mici, foarte mari, invalide etc.)

Problemele pe care le detectează boundary testing sunt, de obicei:

- folosirea incorectă a operatorilor relaționali (mai mic, mai mare etc.)
- folosirea de valori constante inițializate incorect
- erori de calcul care ar putea realiza overflow sau wrap-around în cazul conversiei între diverse tipuri de date

Link-uri utile

- Boundary testing isn't guessing at numbers [http://blogs.msdn.com/imtesty/archive/2008/11/04/boundary-testing-isn-t-guessing-at-numbers.aspx]
- boundary values software testing [http://www.geocities.com/xtremetesting/BoundaryValues.html]
- Boundary Value Testing [http://www.win.tue.nl/~mousavi/testing/1.pdf]
- Boundary Value Analysis [http://www.cs.swan.ac.uk/~csmarkus/CS339/dissertations/NeateB.pdf]

Unit testing

Unit testing este o metodă folosită pentru a testa fiecare componentă a unui proiect. O *unitate* este cea mai mică componentă a unei aplicații. În mod ideal modulele de test sunt independente unele de celelalte. Pentru fiecare *unitate* se fac teste separate.

Există și o abordare **Test Driven Development - TDD** în care se scrie testul pentru unitate înainte de scrierea codului.

Stubs

O problemă des întâlnită în testarea proiectelor este testarea unei părți a proiectului înainte ca alte părți să fie gata. Se pot folosi pentru asta interfețe, numite **Stubs**, care simulează funcțiile de bază ale obiectului respectiv fără să efectueze și teste de integritate a datelor sau ale fluxului logic al problemei. Ele sunt des folosite în cursul dezvoltării unităților proiectului care depind de obiectul simulat. Accentul este pus pe verificarea răspunsului dorit / schimbării de stare.

Mockup

Mockups sunt tot implementarea unor interfețe care testează mai aprofundat funcțiile necesare. Ele simulează spre exemplu funcționarea unui server pentru a putea testa facilitățile clientului și testează de asemenea autentificarea clientului înainte ca acesta să poată efectua anumite tranzacții. Pentru o utilizare mai facilă se recomandă folosirea interfețelor și utilizarea lor în funcția de testare. O implementare pentru testare este o implementare care conține numai cod de test și imită cât mai bine funcționarea viitorului obiect. Accentul este pus pe verificarea comportamentului. Mockup-urile sunt utile în multe situații precum: * cazul când obiectul în sine nu există * obiectul real/funcția reală ia foarte mult timp să ruleze * obiectul real este prea dificil de pus în funcțiune * funcția reală returnează valori nedeterministe și se dorește testarea comportării cu toate valorile limită * funcția reală necesită interacțiunea cu utilizatorul și nu se poate folosi în teste automate

Important este ca atunci când se folosesc obiecte pentru simulare, trebuie să se țina cont de faptul că obiectul trebuie să simuleze cât mai bine realitatea. Există și facilități implementate pentru folosirea mockup-urilor în .NET precum NMock, POCMock, .NET Mock Object.

Regression testing

“Also as a consequence of the introduction of new bugs, program maintenance requires far more system testing per statement written than any other programming. Theoretically, after each fix one must run the entire batch of test cases previously run against the system, to ensure that it has not been damaged in an obscure way. In practice, such regression testing must indeed approximate this theoretical idea, and it is very costly.” – Fred Brooks, The Mythical Man Month (p 122)

Regression testing implică verificarea ca odată cu avansarea în proiect să nu se piardă funcționalități deja implementate, sau să se genereze erori noi.

Cea mai simplă și eficientă metodă de regression testing este să se păstreze toate testele într-un batch care să se ruleze periodic, astfel orice bug nou va fi remarcat imediat și poate fi remediat. Desigur, asta implică ca testele respective să poată fi rulate automat.

Fault injection

'Fault injection' este o metodă de testare software care implică generarea de input-uri care să ducă programul pe căi (în general de error handling) care altfel ar fi parcurse foarte rar în decursul unei testări normale, îmbunătățind astfel foarte mult code coverage-ul.

Există atât software cât și hardware fault injection.

HWIFI(Hardware Implemented Fault Injection)

Există încă din 1970, și implică crearea de scurtcircuite pe placă, generând astfel erori.

SWIFI(Software Implemented Fault Injection)

Se împarte în două mari categorii # Compile time injection # Run time injection

Compile time injection

Modificarea de linii de cod la compilare pentru a genera comportamente eronate. Ex: a++ poate fi modificat în a-;

Run time injection

- Coruperea spațiului de memorie al procesului
- Interceptarea syscall-urilor și introducerea de erori în ele
- Reordonarea, coruperea și distrugerea pachetelor de pe rețea.

Platforme de testare

- Java: una din cele mai cunoscute platforme de testare este JUnit [http://www.junit.org/] pentru care găsiți aici [http://www.cavdar.net/2008/07/21/junit-4-in-60-seconds/] un tutorial.
- C#: pentru cam toate platformele .NET există NUnit [http://www.nunit.org/index.php]. Un tutorial gasiți la adresa http://www.nunit.org/index.php?p=quickStart&r=2.4 [http://www.nunit.org/index.php?p=quickStart&r=2.4]
- Python unittest [http://docs.python.org/library/unittest.html] pentru Python.

Testarea interfețelor grafice

Există mai multe utilitare pentru testarea automată a programelor cu interfețe grafice (o listă mai detaliată aveți aici [http://en.wikipedia.org/wiki/List_of_GUI_testing_tools]).

AutoIt

AutoIt [http://www.autoitscript.com/autoit3/] este un limbaj de programare asemănător Visual Basic cu un compilator ce rulează pe Windows și care permite (printre altele):

- apelul unor funcții din DLL-uri Win32
- execuția de aplicații (consolă/GUI)
- creare de interfețe GUI (ferestre de mesaje, atenționare, de introducere de date, etc.)
- manipulare sunete
- simulare mișcări de mouse și apăsare taste și combinații de taste
- manipulare ferestre și procese
- manipulare elemente în cadrul unei ferestre

Scripurile pot fi compilate sub forma unor executabile Win32.

Două tutoriale de AutoIt: interacțiune cu notepad [http://www.autoitscript.com/autoit3/docs/tutorials/notepad/notepad.htm] și instalare winzip [http://www.autoitscript.com/autoit3/docs/tutorials/winzip/winzip.htm]

Abbot

Abbot [http://abbot.sourceforge.net/doc/overview.shtml] este o platformă de testare automată a aplicațiilor GUI scrise în Java. Testele sunt scrise sub forma unor unit-test-uri. Mai multe detalii pe site-ul proiectului.

Code coverage, Code profiling

Deși folosite în special pentru optimizări și pentru identificarea bottleneck-urilor din sistem, utilitarele de tip code-coverage și code-profiling pot fi folosite pentru detectarea anumitor tipuri de probleme precum bucle infinite, sincronizare ineficientă etc.

Code coverage

Utilitarele de tipul code coverage sunt folosite în procesul de testare a programelor pentru inspectarea unei părți cât mai mari a programului. Diversele tipuri de mecanisme de tip code coverage sunt folosite pentru a determina ce funcții sunt acoperite la o rulare, ce instrucțiuni sunt apelate, ce fluxuri de execuție sunt parcurse.

Programele folosesc opțiuni speciale de code-coverage. Cu ajutorul acestor opțiuni se pot determina funcțiile sau instrucțiunile des (sau rar) folosite și oferă o imagine a nivelului de testare a anumitor părți dintr-un program.

În general, utilitarele de code coverage sunt privite ca utilitare pentru depanare automată și sunt folosite, de obicei, de inginerii de testare. Depanarea efectivă, cu utilitare de debugging specializate, este realizată, în general de dezvoltatorii care au cunoștință de codul inspectat.

Code profiling

Profilerele sunt utilitare care prezintă informații referitoare la execuția unui program. Sunt utilitare care intră în categoria “dynamic analysis” spre deosebire de alte programe care intră în categoria “static analysis”.

Profilerele folosesc diverse tehnici pentru colectarea de informații legate de un program. De obicei se obțin informații de timp petrecut în cadrul unei funcții (nivel ridicat) sau numărul de cache miss-uri, TLB miss-uri (nivel scăzut).

În general, un program care este “profiled” este instrumentat astfel încât, în momentul rulării, să ofere la ieșire informațiile utile dorite. Spre exemplu, pentru a folosi opțiunile gprof, se folosește opțiunea `-pg` transmisă gcc.

Software Quality Assurance (SQA)

- ciclu de asigurare a calității produsului software
- se desfășoară de-a lungul întregului ciclu de dezvoltare a produsului software, pe care **îl îndrumă, monitorizează, auditează, evaluează și coordonează**
- complementează procesul de dezvoltare a proiectului
- (în mediile enterprise) este realizat de către un grup de resurse umane dedicate asigurării calității.

Domeniul QA

- metodologia QA a fost introdusă inițial în cadrul fabricilor, unde și-a demonstrat aportul la succesul producției de calitate
- ulterior, s-a dorit adoptarea QA la mediul dezvoltator de aplicații software
- *Problema*: produsul software nu este palpabil, deci măsurarea funcțiilor, a beneficiilor și a costurilor sunt mai dificil de determinat
- *Soluția*: s-au adăugat concepte precum quality attributes (metrici software, cerinte functionale si non-functionale) pentru a putea măsura calitatea produsului software.

Cel mai cunoscut *model de atribute* ale produselor software de calitate este **FURPS+ model**, care cuprinde următoarele atribute:

1. **funcționalitate** (*functionality*)
   - cuprinde toate cerințele funcționale ale sistemului așa cum au fost stabilite în SRS
2. **utilizabilitate** (*usability*)
   - cuprinde cerințe legate de ușurința de folosire a interfeței sistemului de către utilizator (interfața trebuie să fie estetică, intuitivă, ergonomică, consistentă, accesibilă și celor cu disabilități)
3. **încredere** (*reliability*)
   - cuprinde cerințe legate de precizia răspunsurilor, de disponibilitatea și toleranța la defecte a sistemului
4. **performanță** (*performance*)
   - cuprinde cerințe legate de timpii de lucru, precum timp de răspuns (response time), timp de revenire (recovery time), timp de lansare (startup time)
5. **avantaje suplimentare** (*supportability*)
   - cuprinde cerințe legate de ușurința testării (testability), a mentenanței (maintainability), a configurării (configurability), de gradul de scalare (scalability) și compatibilitate (compatibility)

și de oferire de multiple traduceri ale conținutului interfeței cu utilizatorul.

- "+" se referă la faptul că la acest model se pot adăuga și alte cerințe de calitate personalizate.

Au apărut **standarde** dedicate produselor software, de tipurile:

- **standarde de managementul calității** (*Quality management*)
  - *ISO 9000 – 3* Quality Management and Quality Assurance Standards – Part 3: Guidelines for the application of 9001 to the development, supply, installation and maintenance of computer software
- **standarde de documentare** (*Documentation Standards*) specifică formatul și conținutul documentelor de planificare și control și a documentației produsului
  - **standarde de planificare** (*Producing plans*): *IEEE Std1059-1993* Guide for Software Verification and Validation Plans
- **standarde de Project Management**
  - *General project management*: *IEE Std 1058.1-1987* Standard for Software Project Management Plans
- **standarde de Software Engineering**
  - *Software Project Lifecycle*: *ISO/IEC WD 15288* System Life Cycle Processes
  - *Software Project Requirements*: *IEEE Std 1233-1996* Guide for Developing System Requirements Specifications
  - **standarde de proiectare** (*Design Standards*) specifică reguli și metode de proiectare a soluției pornind de la specificații și formatul și conținutul documentelor de proiectare
  - **standarde de codare** (*Code Standards*) specifică limbajele de programare alese, convențiile de stil, reguli de construire a interfețelor și modul de comentare a surselor.

Ulterior, s-au definit **proceduri**:

- sunt succesiuni de pași ce trebuie urmați pentru a împlini un proces (ex: testare, configuration management).

Activități SQA

Activitățile principale SQA sunt:

- asigură că standardele și procedurile de calitate alese pentru a fi urmate în proiect sunt *adecvate* la specificul proiectului (stabilirea lor este critică pentru că standardele oferă criteriile de evaluare a produsului și procedurile criteriile de comparare pentru procesele de dezvoltare și control)
- asigură că standardele și procedurile alese sunt *bine documentate* pentru că activitățile de monitorizare, audit și evaluare se bazează pe ele
- monitorizarea proceselor de dezvoltare și control *în raport cu procedurile de calitate* (pașii urmați coincid cu cei ai procedurilor?)
- evaluarea proiectului *în raport cu standardele de calitate*.

Activitățile de evaluare și monitorizare au loc în cadrul **auditurilor**. *Auditul* este *tehnica SQA de bază* folosită pentru verificarea calității produsului

- întocmește *rapoarte de audit* care sunt predate management-ului, ce afirmă gradul de implementare a calității și oferă *recomandări* pentru implementarea / sporirea calității în faza următoare a proiectului.

Există **activități SQA specifice fazelor** ciclului de viață al proiectului

1. *faza de inițiere*

- SQA intervine în redactarea și revizuirea planului de management
- asigură că standardele și procedurile alese sunt potrivite, clare și pot servi ca bază de auditare

2. *faza cerințelor software*
   - SQA intervine în revizuirea specificațiilor
   - asigură că specificațiile sunt clar exprimate, sunt categorisite corect în cerințe funcționale și non-funcționale (de interfață, de performanță, etc.), acoperă toate cerințele utilizatorului, pot fi măsurate
3. *faza de proiectare*
   - SQA intervine în revizuirea documentelor de proiectare
   - asigură că toate cerințele software au fost traduse corect în componente software în raport cu standardele și procedurile de proiectare
4. *faza de dezvoltare*
   - SQA asigură că soluția este dezvoltată în concordanță cu documentul de proiectare și cu standardele și procedurile de codare
5. *faza de testare*
   - SQA asigură respectarea standardelor și a procedurilor de testare
   - anunță sfârșitul procedurii de testare a cerințelor funcționale
6. *faza de livrare*
   - SQA verifică și declară respectarea cerințelor non-funcționale
   - anunță că produsul final este gata de livrare
7. *faza de mentenanță*
   - SQA intervine pentru a asigura că subciclurile de dezvoltare respectă normele de calitate

SQAP (Software Quality Assurance Plan)

În general, scopul unei organizații este crearea unui **SQAP** care să asigure nivelul dorit de calitate a produsului.

Din standardul IEEE 730-1998 [http://ieeexplore.ieee.org/document/720573/], structura unui SQAP conține următoarele secțiuni:

1. Scopul documentului (Purpose)
2. Documente referite
3. Management
4. Documentație
5. Standarde, practici, convenții, metrici
6. Revizii și audituri
7. Managementul riscului
8. Raportarea problemelor și acțiuni de corecție
9. Utilitare, tehnici și metodologii
10. Controlul furnizorului
11. Training
12. Colectarea înregistrărilor, mentenanța

Un exemplu de document SQAP puteți găsi aici [http://people.cs.ksu.edu/~ganti/mse_sqa.htm].

Terminarea unui proiect

Contexte de terminare

Un proiect se poate finaliza în următoarele contexte:

- **s-a terminat** (cazul cel mai fericit - proiect de *succes*)
- **a fost oprit** din motive (proiect *eșec*):

- **de afaceri** (apar oportunități noi de business care ar genera un profit semnificativ mai mare decât proiectul actual)
- **de cost** (bugetul alocat proiectului a fost epuizat și nu mai există surse de finanțare)
- **de calitate** (calitatea proiectului este foarte redusă, deci investiția nu mai merită)
- **tehnice** (între timp, au fost lansate soluții mai viabile decât cea a proiectului actual, tehnologiile utilizate de proiect au devenit demodate)
- **de timp** (timpul alocat proiectului s-a încheiat)

Feluri de terminare a unui proiect:

- *Extinctie* (proiectul se termină prin succes sau eșec și, pe viitor, nu se va mai lucra la proiect)
- *Terminare prin adiție* (proiectul se termină cu succes și, pe viitor, echipa care a dezvoltat proiectul se va ocupa de mentenanța produsului)
- *Terminare prin integrare* (proiectul se termină cu succes și resursele sunt reintegrate în alte proiecte ale companiei – modul cel mai frecvent)

Checklist-ul terminării proiectului

- cuprinde *activitățile rămase* din cadrul proiectului
- cuprinde *activitățile colaterale* ce trebuie realizate pentru a încheia proiectul în conformitate cu procedurile actuale:
  - completarea și distribuirea rapoartelor de performanță
  - completarea și distribuirea documentației sistemului
  - completarea auditurilor de calitate
  - completarea auditurilor de vânzare
  - revizia sistemului împreună cu help desk
  - întâlnirea cu staff-ul operațional care urmează să preia software-ul (în vederea mentenanței)
  - împlinirea cerințelor de training ale staff-ului operațional
  - primirea sign-off-ului de la staff-ul operațional
  - oferirea detaliilor despre performanța membrilor proiectului (evaluarea echipei și evaluările individuale ale membrilor și a PM-ului)
  - primirea sign-off-ului de acceptare de la client și acceptarea formală a tuturor livrabilelor proiectului.

În cazul în care se folosește un sistem de bug/issue tracking, trebuie ca *toate issue-urile să fie rezolvate* înainte de finalizarea proiectului chiar dacă ele nu au fost incluse în specificații.

În cazul în care nu se poate acest lucru, cei care predau proiectul trebuie să se asigure că problemele rămase deschise sunt de prioritate scăzută și nu au un impact major asupra funcționalității.

Procesul de terminare

- se decide terminarea proiectului
- se obțin aprobările necesare (de la clienți, sponsori)
- se comunică decizia părților interesate
- se realizează activitățile rămase din proiect și cele colaterale
- se face post-performance analysis
- se publică raportul final
- se sărbătorește terminarea proiectului
- se reasignează resursele (persoane și echipamente)
- se efectuează închiderea financiară și administrativă

Post-Performance Analysis (PPA)

Încheierea unui proiect nu presupune numai livrarea produsului, ci și **oportunitatea de a învăța** din această experiență pentru îmbunătățirea contribuțiilor la proiectele viitoare.

Recomandări:

- analiza finală a proiectului să se bazeze pe *metrici* (așa se pot cuantifica acurat rezultatele);
- exemplu de metrici:
    1. **calitatea produsului**:
        a. *Defects per KLOC* (Kilo Line Of Code): numărul de buguri per KLOC, unde **KLOC** este unitatea de măsură pentru codul produsului
        b. *MTTC* (Mean Time To Change): cât timp este necesar pentru a localiza eroarea, a proiecta repararea, a o implementa si testa
        c. *threat probability*: probabilitatea atacurilor ce exploatează vulnerabilitățile produsului pe o perioadă de timp
        d. *security probability*: probabilitatea eliminării vulnerabilităților pe o perioadă de timp
    2. **calitatea activităților QA**:
        a. *Test Coverage*: numărul de unități testate (KLOC/FP) / dimensiunea sistemului, unde **FP** (function point) este unitate de măsură pentru codul produsului din perspectiva funcționalității de business oferite
        b. *Number of tests per unit size*: numărul de teste per KLOC/FP
        c. *Cost to locate defect*: costul de testare / numărul de defecte detectate
    3. ***Quality of Testing***: E / (E + D), unde E este numărul de erori identificate înainte de livrarea produsului, D numărul de erori identificate după livrare
        a. *System complaints*: numărul de plângeri din partea clienților / numărul de plângeri rezolvate
        b. *Test Planning Productivity*: numărul de teste proiectate / efortul pentru proiectarea și documentarea testelor
        c. *Test Execution Productivity*: numărul de cicluri de testare executate / efortul de testare
        d. *Test efficiency*: numărul de teste solicitate / numărul erorilor sistemului
- să se determine *cauzele* ce au condus la valorile finale ale metricilor
- să se rețină și să se înțeleagă aceste cauze, să se determine ce *posibilități* ar fi existat/ar exista pentru a evita producerea acestor cauze, ce posibilități sunt de stimulare a producerii a acestor cauze
- să se colecteze *valorile reutilizabile* care au fost produse în cadrul proiectului (proceduri, checklisturi, guidelines) și să se facă publice

PPA:

- este *analiza situației finale* a proiectului în vederea concluzionării asupra factorilor de succes și a modalităților de stimulare a lor, precum și asupra factorilor de insucces și a modalităților de prevenire, rezolvare a efectelor lor
- este *analiza ce stimulează procesul de învățare* din experiența proiectului
- este recomandat să se realizeze periodic (imediat după milestone-uri) pentru a putea folosi concluziile analizei chiar in faza următoare

Procesul PPA are următorii pași:

1. invitarea echipei la analiză (propunând ca fiecare membru să reflecteze asupra factorilor de succes/insucces și să vină cu propuneri de îmbunătățire - se trimite un chestionar de analiză)
2. colectarea individuală a feedback-ului fiecărei persoane implicate în proiect (individual pentru a înțelege întreaga panoramă a proiectului)
3. realizarea unei întâlniri a echipei în vederea concluzionării asupra: ce și cum s-a întâmplat, în ce moduri ar fi putut fi prevenite/soluționate problemele?
4. publicarea și arhivarea sumarului PPA.

În general, se folosește **un chestionar** pentru colectarea informațiilor. De obicei, acesta are conținut diferit între team leaderi și team memberi.

Câteva întrebări de chestionar sunt cele de mai jos:

1. Identificați lucrurile care au mers bine în cadrul acestui proiect.
2. Identificați lucrurile care au mers prost în cadrul acestui proiect.
3. Ce evenimente neprevăzute au afectat pozitiv sau negativ desfășurarea proiectului?
4. În cadrul acestui proiect, ce lucruri ați face diferit dacă ar fi repornit?
5. Descrie un lucru pe care tu l-ai fi putut realiza personal pentru a îmbunătăți calitatea produsului obținut din cadrul acestui proiect.
6. Ai fost informat despre ceea ce se așteaptă de la tine în cadrul acestui proiect?
7. Au fost rolurile membrilor echipei definite clar?
8. Echipa a făcut tot ce se putea face pentru a duce la bun sfârșit acest proiect?
9. Cum s-a comportat echipa în ansamblul său?
10. Care a fost componenta cea mai satisfăcătoare din cadrul proiectului?

Un exemplu complet de chestionar găsiți aici [http://michaelgreer.biz/?p=161]

Bibliografie

- http://www.sqatester.com/methodology/PositiveandNegativeTesting.htm [http://www.sqatester.com/methodology/PositiveandNegativeTesting.htm]
- http://www.opensourcetesting.org/testmgt.php [http://www.opensourcetesting.org/testmgt.php]
- http://www.sqatester.com/documentation/testcasesmpl.htm [http://www.sqatester.com/documentation/testcasesmpl.htm]
- http://www.computerhistory.org/tdih/September/9/ [http://www.computerhistory.org/tdih/September/9/]

Exerciții (50 min)

- Se dau următorul patch: patch [https://curl.haxx.se/CVE-2016-8616.patch] și următorul tweet: tweet [https://i.imgur.com/KHaCdxA.png]. Studiati-le timp de 5 minute. Vor fi importante pentru testul nostru Kahoot [https://kahoot.com/schools-u/] :) **Nota**: fiecare răspuns corect valorează un punct, se acordă un punct pentru prezență. Punctajul total aferent laboratorului curent este de **11p**.

Lucru la proiect (50 min)

- Descrieți sumar conținutul unui SQAP pentru proiectul vostru.

  - Parcurgeți secțiunea Software Quality Assurance Plan [https://ocw.cs.pub.ro/courses/mps/laboratoare/laborator-09#sqap_software_quality_assurance_plan]

- Pregătiți un checklist pentru terminarea proiectului vostru. Argumentați!

  - Parcurgeți secțiunea Terminarea unui proiect

- Răspundeți la întrebările de la procesul PPA pentru proiectul vostru.

  - Parcurgeți secțiunea Post-Performance Analysis.

Stabiliti ce mai aveti de facut pentru finalizarea proiectelor.